

Assignment 1 Technical Report (Part 1)

1. Scope and Data Sources

This document reports and interprets the performance results of three dense matrix multiplication implementations:

1. Exercise 1: OnMult (naive loop order)
2. Exercise 2: OnMultLine (loop reordering for locality)
3. Exercise 3: OnMultBlock (cache blocking/tiling)

2. System Specifications

- Compiler context for C++ runs: optimized build (O2) for the benchmark binary.
- Cache line size assumption: 64 bytes (typical x86_64), which corresponds to 8 double-precision values per line.
- Measured counters: elapsed time, instructions, cycles, IPC, cache references/misses, L1 D-cache loads/misses, LLC loads/misses.

Why this matters: the behavior of these kernels is dominated by memory hierarchy effects, so cache geometry and data layout are first-order performance factors.

3. Methodology

- Matrix sizes: 1024 to 10240 (depending on algorithm runtime feasibility).
- Primary KPI: elapsed time.
- Secondary KPIs: speedup, GFLOPS estimate, IPC, and miss rates.
- Comparison strategy:
 1. Algorithmic progression (OnMult -> OnMultLine -> OnMultBlock).
 2. Cross-size scalability trends.
 3. Block-size sensitivity for the tiled kernel.

4. Exercise 1: OnMult (Naive i-j-k)

4.1 Algorithmic behavior

OnMult uses the textbook triple loop with cubic complexity, but memory access is not cache-friendly for matrix B. In row-major storage, $B[k][j]$ with k as the inner progression leads to large strides across memory, which weakens spatial locality and increases cache miss pressure.

4.2 Perf-centered interpretation

- Elapsed time rises from about 2.30 s ($N=1024$) to 153.01 s ($N=3072$).
- GFLOPS decreases from about 0.93 to 0.38, showing that larger problems are increasingly memory-limited.
- The IPC starts low and drops even further as the matrix grows (from 0.65 to 0.27). This happens because the CPU spends most of its time waiting for data to arrive from the main memory, leading to many idle

cycles.

- L1 miss rate is very high (about 0.49 -> 0.87), which explains why arithmetic units are often waiting for data.

This kernel becomes memory bound early, since poor locality causes repeated long-latency data fetches that prevents the CPU from finishing instructions quickly because it is waiting for data from memory.

5. Exercise 2: OnMultLine (i-k-j)

5.1 Why i-k-j is faster

In row-major order, contiguous addresses correspond to contiguous columns within the same row. The i-k-j traversal improves locality in two critical ways:

- $A[i][k]$ is read once and reused across the whole inner j loop.
- $B[k][j]$ is traversed across j, which means contiguous accesses in memory.

With a 64-byte cache line, each line fetch brings 8 doubles. Therefore, when j advances sequentially, several consecutive multiplications consume values already present in L1/L2. This converts many potential RAM accesses into cache hits.

In performance terms, the kernel shifts from strongly memory bound toward a better balanced regime, with significantly higher compute utilization.

5.2 Measured evidence

- Elapsed time at $N=1024$: 0.336 s vs 2.300 s in OnMult.
- Elapsed time at $N=3072$: 13.34 s vs 153.01 s in OnMult.
- Speedup vs OnMult ranges from 6.85x to 11.47x for common sizes.
- IPC increases drastically (around 5.23 at $N=1024$ and around 3.25 to 3.71 at larger N), proving better pipeline feed and fewer long stall periods.

6. Exercise 3: OnMultBlock (C++)

6.1 Hardware-aware rationale

Blocking partitions the full problem into smaller sub-matrices to increase temporal reuse in cache.

For our tests, we calculated the memory footprint for one tile (using 8 bytes for double precision):

- $b=128$: 128 KiB per tile
- $b=256$: 512 KiB per tile
- $b=512$: 2 MiB per tile

During blocked multiplication, active working data involves tiles from A, B, and C. The $b=256$ regime often provides a strong trade-off: enough locality improvement without overinflating the working set. In practical terms, this is consistent with sub-matrix residency in private cache levels (typically L2) and reduced pressure on shared LLC (L3), which lowers cache thrashing. In this dataset, that behavior is visible for most sizes, while $b=512$ becomes preferable only at the largest matrices.

6.2 Measured evidence and cache link

- Best block is mostly 256 for $N=1024$ to 6144 .
- Best block changes to 512 for $N=8192$ and $N=10240$.
- Speedup vs OnMultLine remains positive across all tested points (about 1.02x to 1.77x).
- Speedup vs OnMult reaches up to about 17.25x.
- IPC remains high (around 4.5 to 4.8 in many cases), supporting the claim that the blocked kernel sustains much better execution efficiency.

The blocked kernel mitigates cache thrashing and improves data reuse distance, allowing the computation to remain closer to a compute-efficient regime even as N grows.

7. C++ vs Python (Exercises 1 and 2)

7.1 Empirical comparison

Using Python's onMult and onMultLine against the C++ measurements:

- Exercise 1 (OnMult): Python is roughly 16x to 40x slower, depending on N .
- Exercise 2 (OnMultLine): Python is roughly 112x to 157x slower.

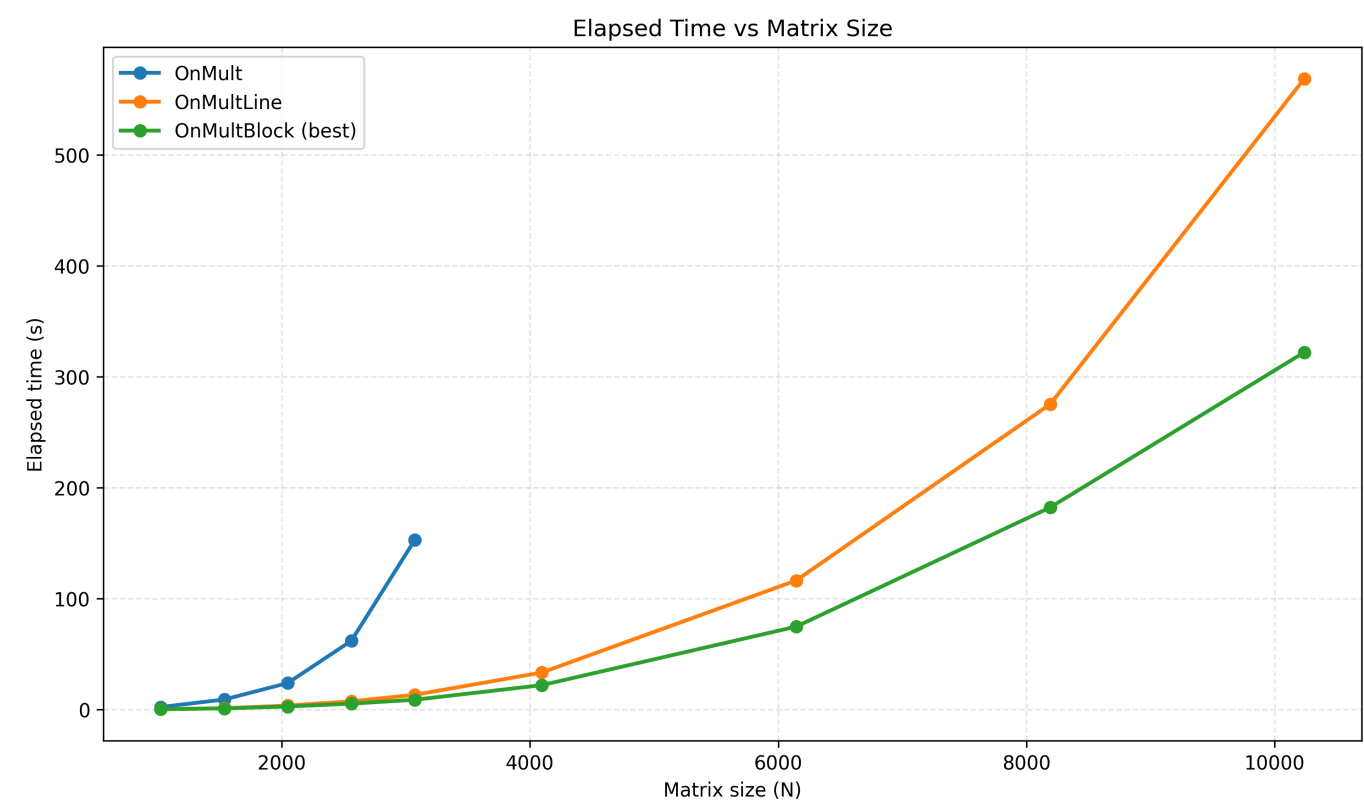
7.2 Root causes

- Interpreter overhead in every loop iteration.
- Dynamic object model overhead (including boxing/unboxing and reference management costs for numeric operations).
- Lack of low-level compiler optimizations available in C++ O2 builds.
- Reduced automatic exploitation of SIMD/vector instructions in pure Python loops.

Practical consequence: algorithmic optimization helps in both languages, but optimized C++ can exploit hardware features (instruction scheduling, vectorization, register allocation) that pure Python loops cannot match.

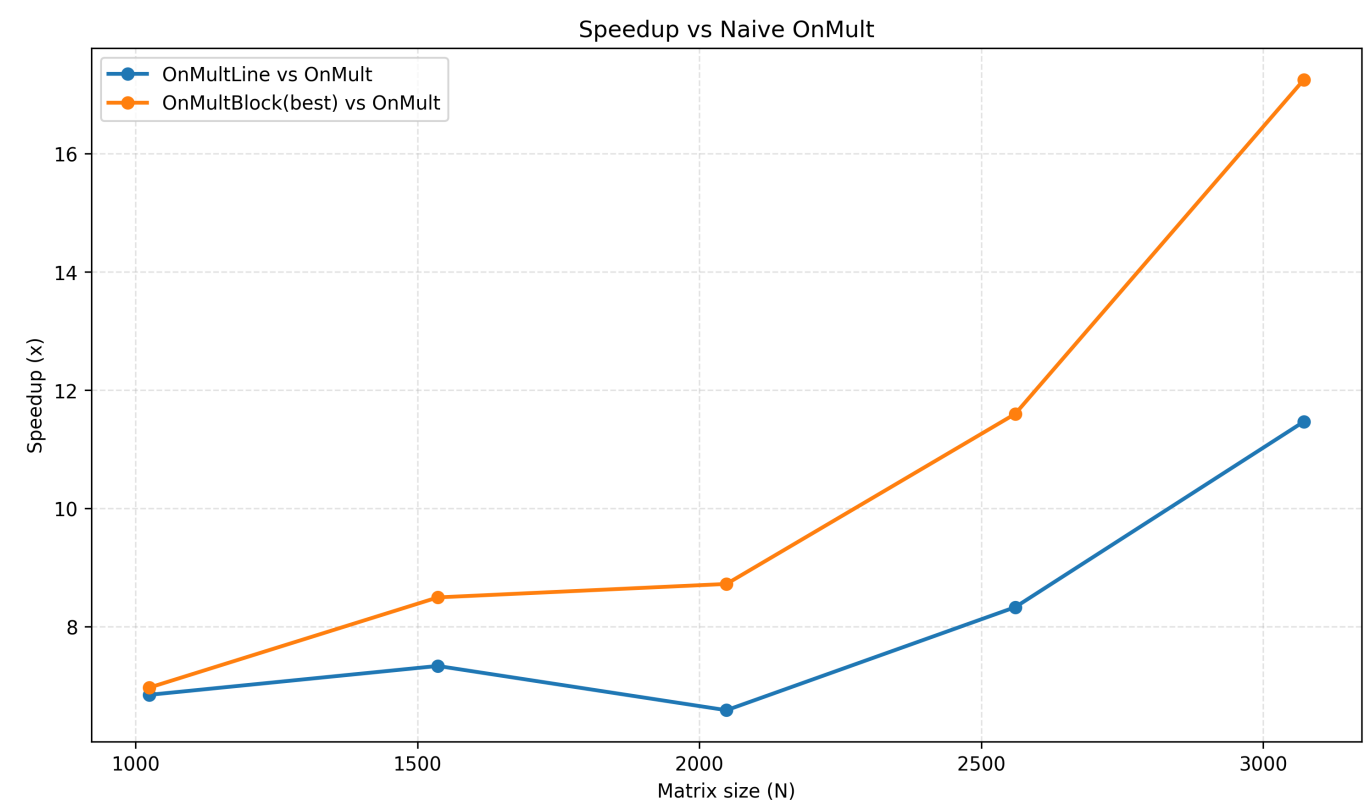
8. Figures and Technical Captions

Figure 1 - End-to-end elapsed time



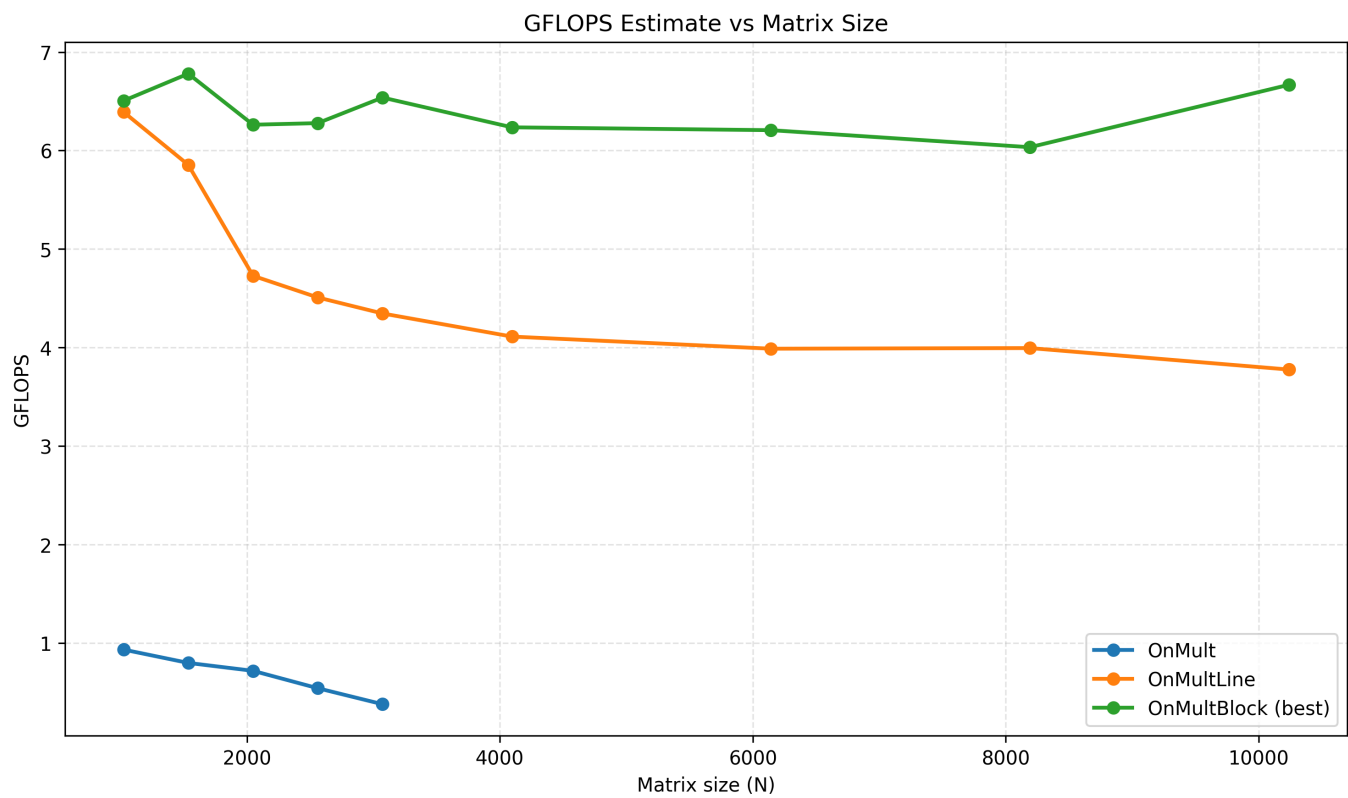
Caption: The figure shows a clear divergence in execution time growth. OnMult exhibits the steepest cubic-like escalation, OnMultLine reduces the slope through better access order, and OnMultBlock remains the most scalable due to cache-aware tiling.

Figure 2 - Speedup over naive baseline



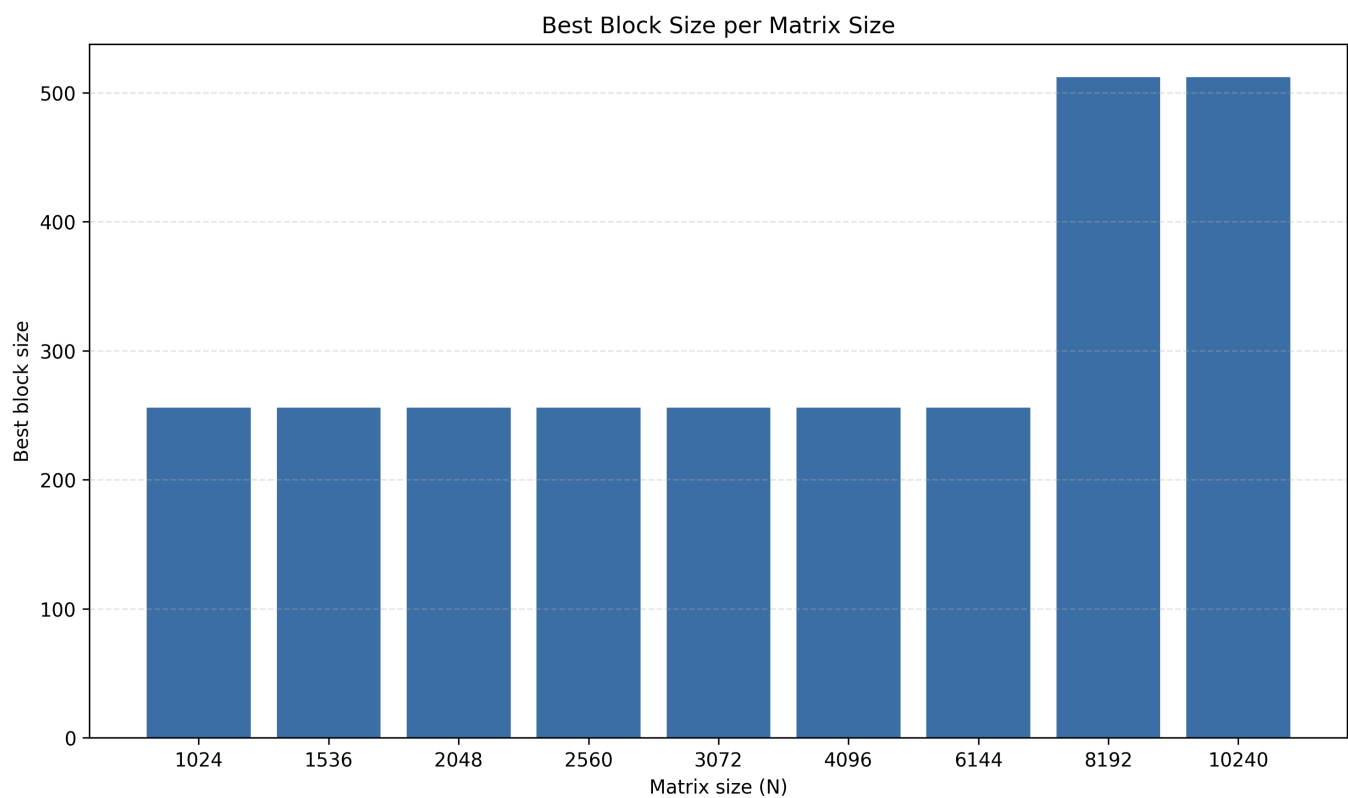
Caption: Both optimized kernels accelerate relative to OnMult, but OnMultBlock consistently dominates OnMultLine, confirming that locality plus tiling outperforms locality-only loop reordering.

Figure 3 - Throughput trend (GFLOPS)



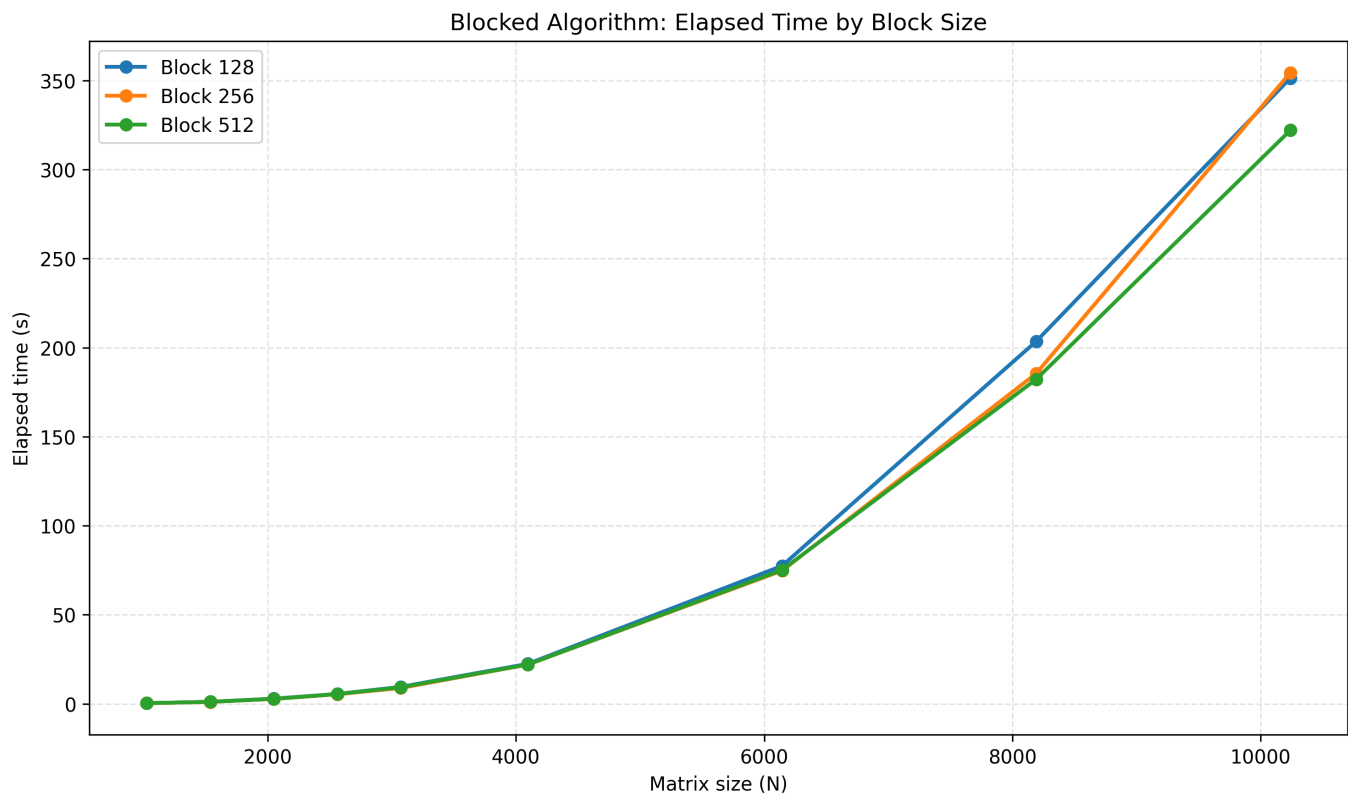
Caption: OnMultBlock sustains the highest and most stable throughput across sizes. OnMultLine remains intermediate, while OnMult loses efficiency quickly as memory stalls increase.

Figure 4 - Best block size by matrix size



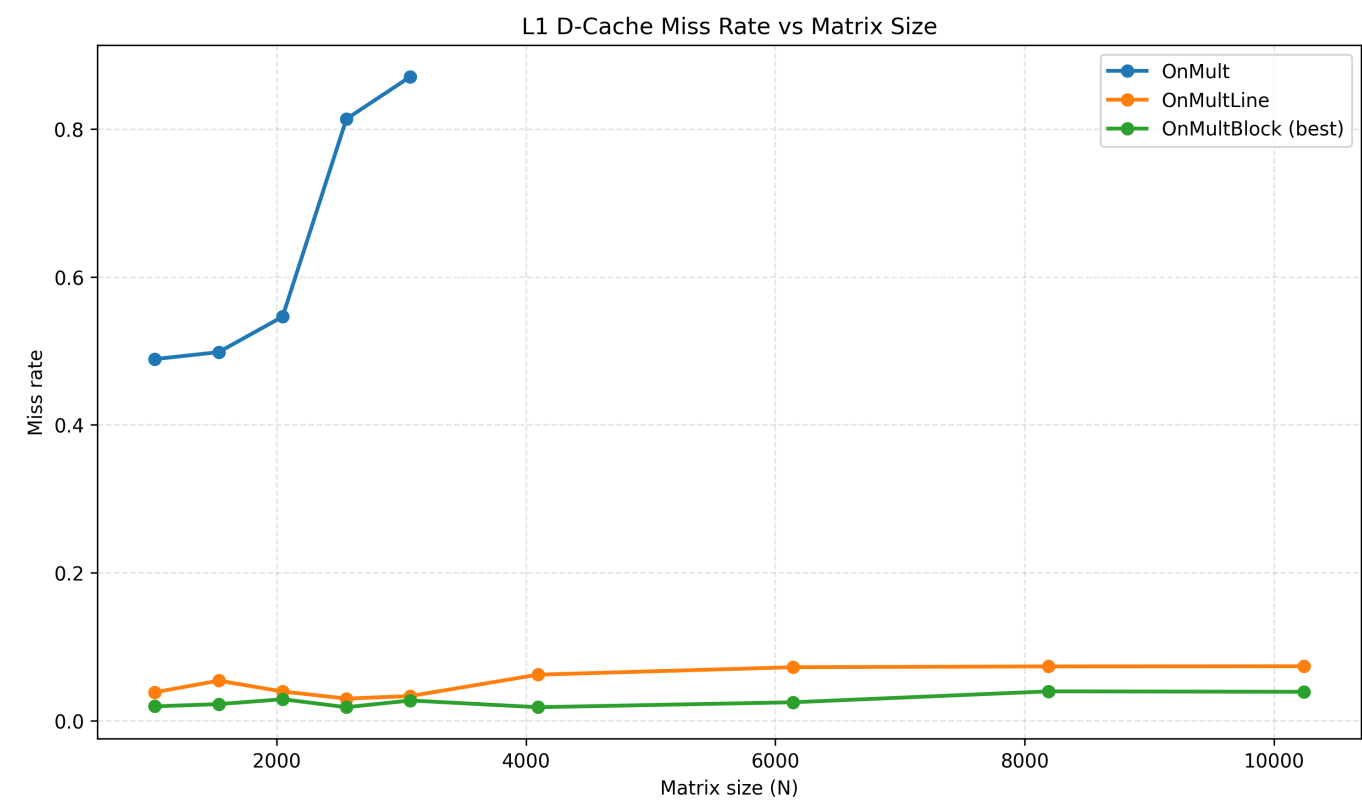
Caption: The optimal block size is mostly 256 for small/medium-large problems, switching to 512 at the largest dimensions. This indicates a size-dependent locality/overhead balance.

Figure 5 - Block sensitivity (elapsed time)



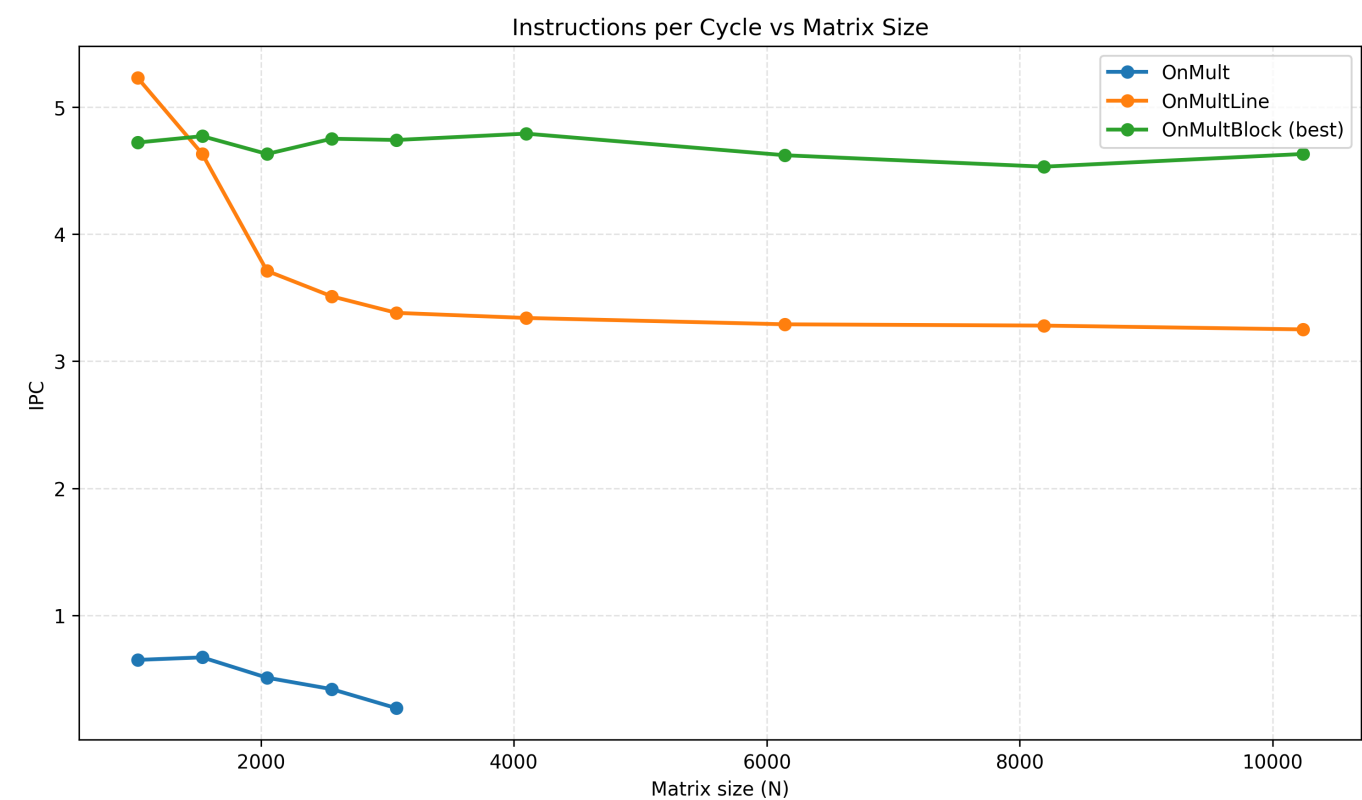
Caption: Block-size choice materially affects execution time. No single block is universally optimal; the best point shifts with problem size and cache-pressure regime.

Figure 6 - L1 miss-rate comparison



Caption: OnMult has the highest miss-rate growth, directly explaining its low IPC. OnMultLine and OnMultBlock keep misses much lower, and this correlates with their higher instruction throughput.

Figure 7 - IPC trend across algorithms



Caption: IPC is the strongest microarchitectural confirmation of optimization quality in this study: low and degrading for OnMult, substantially higher for OnMultLine and OnMultBlock.

9. Conclusion

1. Loop order is not a micro-optimization detail; it fundamentally changes memory behavior and therefore runtime.
2. OnMultLine demonstrates how row-major-aware traversal increases spatial locality and reduces memory bottlenecks.
3. OnMultBlock further improves locality by explicitly shaping the working set to cache-friendly tiles.
4. The observed best block sizes are consistent with cache-capacity and reuse-distance reasoning.
5. IPC and miss-rate trends provide hardware-level evidence that the achieved speedups are structural, not accidental.